

QUY TRÌNH PHÁT TRIỂN PHẦN MỀM

Từ ý tưởng đến sản phẩm hoàn chỉnh: Hành trình của Software Development Life Cycle (SDLC)

SDLC là gì?

Quy trình phát triển phần mềm là tập hợp các hoạt động, phương pháp và bước đi có tính hệ thống nhằm xây dựng, triển khai và bảo trì một sản phẩm phần mềm.

Tại sao cần SDLC?

Viết code chỉ chiếm 20-30% dự án. Thiếu quy trình dẫn đến "Code first, think later": sai lệch yêu cầu, lãng phí nguồn lực, lỗi dây chuyền và vỡ trận.

Các Giai đoạn Chính

- 1. Phân tích Yêu cầu**
Tìm hiểu và đặc tả mong muốn của khách hàng.
- 2. Thiết kế**
Vẽ sơ đồ, kiến trúc hệ thống, giao diện (UI/UX).
- 3. Lập trình**
Chuyển thiết kế thành code thực thi.
- 4. Kiểm thử**
Tìm lỗi, đảm bảo phần mềm chạy đúng và bảo mật.
- 5. Triển khai & Bảo trì**
Đưa sản phẩm vào sử dụng, theo dõi và sửa lỗi phát sinh.

Mô hình Phát triển tinh gọn (Lean Software Development): Nguyên tắc

Nguồn Gốc & Áp Dụng Lean



Lean bắt nguồn từ Hệ thống Sản xuất Toyota (TPS) vào những năm 1940. Mary và Tom Poppendieck đã áp dụng thành công các nguyên tắc này vào Phát triển Phần mềm.

● Toyota TPS

Mục tiêu chính là **tối ưu hóa hoạt động** và **loại bỏ mọi lãng phí** trong quy trình phát triển phần mềm.

Lãng Phí Trong Phần Mềm (Muda)

- **Tính năng thừa:** Không tạo giá trị cho người dùng.
- **Sản phẩm dở dang:** Code chưa được kiểm thử/triển khai.
- **Chuyển giao công việc:** Gây mất mát thông tin.
- **Chờ đợi:** Do tài nguyên hoặc phê duyệt.
- **Lỗi (Defects):** Tốn thời gian và nguồn lực để sửa chữa.

Bảy Nguyên Tắc Cốt Lõi của Lean

- **1. Loại bỏ lãng phí:** Giảm mọi thứ không tạo giá trị.
- **2. Xây dựng chất lượng từ đầu:** Ngăn ngừa lỗi sớm.
- **3. Tạo ra tri thức:** Chia sẻ và học hỏi liên tục.
- **4. Trì hoãn cam kết:** Giữ linh hoạt các lựa chọn.
- **5. Giao hàng nhanh nhất có thể:** Phản hồi và giá trị sớm.
- **6. Tôn trọng con người:** Trao quyền cho đội ngũ.
- **7. Tối ưu hóa toàn cục:** Cải thiện toàn bộ quy trình.

Mô hình Agile: Ứng dụng và nhược điểm

Ứng dụng & Tính phù hợp

- ✔ Phù hợp dự án phức tạp, thay đổi liên tục.
- ✔ Chu kỳ lặp ngắn, phản hồi sớm giúp thích nghi nhanh chóng.
- ✔ Được áp dụng rộng rãi tại Amazon, Spotify, Netflix...

Nhược điểm

- ✘ Thiếu tài liệu thiết kế chi tiết.
- ✘ Khó ước lượng thời gian & chi phí chính xác.
- ✘ Đòi hỏi áp lực tương tác cao giữa các thành viên.

Frameworks Phổ biến

- 🏠 **Scrum:** Cấu trúc rõ ràng, chia việc thành Sprints, vai trò cụ thể.
- 📌 **Kanban:** Linh hoạt, quản lý luồng công việc liên tục bằng bảng.

Khi nào không phù hợp

- ✘ Yêu cầu dự án quá rõ ràng và ổn định từ đầu.
- ✘ Khách hàng thiếu tương tác liên tục, không phản hồi kịp thời.
- ✘ Dự án yêu cầu tính tuân thủ tài liệu nghiêm ngặt.

Mô hình Agile: Nguồn gốc và Giá trị

Agile ra đời từ năm 2001, khi 17 chuyên gia phát triển phần mềm tại Snowbird, Utah, nhằm khắc phục các vấn đề của mô hình truyền thống và định hình lại cách thức phát triển phần mềm hiệu quả hơn.

Vấn đề của Mô hình truyền thống

- Thời gian phát triển quá lâu, sản phẩm đến tay khách hàng muộn.
- Khó điều chỉnh yêu cầu, kém thích ứng với sự thay đổi.
- Tỷ lệ thất bại cao, sản phẩm lỗi thời hoặc không đúng ý người dùng.
- Chi phí và thời gian đội lên khủng khiếp khi phải 'đập đi xây lại'.

4 Giá trị cốt lõi của Agile

- Cá nhân & tương tác > Quy trình & công cụ.
- Phần mềm chạy tốt > Tài liệu đầy đủ.
- Cộng tác với khách hàng > Đàm phán hợp đồng.
- Phản hồi với thay đổi > Bám sát kế hoạch.

12 Nguyên tắc Agile

Tập trung vào sự hài lòng của khách hàng, giao hàng liên tục, chấp nhận thay đổi, cộng tác thường xuyên, hỗ trợ phát triển bền vững, giao tiếp trực tiếp, phần mềm chạy tốt là thước đo chính, phát triển liên tục, xuất sắc về kỹ thuật, đơn giản, đội tự tổ chức, và liên tục cải tiến.

Mô hình Phát triển hướng sử dụng lại (Reuse-based Development): Chi phí và rủi ro

Lợi ích

- Giảm thời gian phát triển (Time-to-market).
- Giảm chi phí nhân công và phát triển.
- Tăng khả năng tái sử dụng các thành phần.
- Giảm thiểu sai sót do thành phần đã kiểm thử.

Rủi ro

- Tính phụ thuộc vào bên thứ ba.
- Chi phí sửa đổi cao hơn viết mới.
- Vấn đề bản quyền (ví dụ: GPL).
- Chi phí tìm kiếm, đánh giá, tích hợp.

Ví dụ thực tế

- **Hệ thống thanh toán:** PayPal/VNPay API (Service Reuse).
- **Bản đồ:** Google Maps API (Service Reuse).
- **Giao diện người dùng:** Bootstrap/Tailwind CSS (Framework Reuse).
- **Nền tảng quản lý nội dung:** WordPress/Shopify (Platform Reuse).

Mô hình Phát triển hướng sử dụng lại (Reuse-based Development): Triết lý

Triết lý cốt lõi: "Ráp nối thay vì xây mới"

Tìm kiếm và tích hợp các thành phần đã kiểm chứng có sẵn thay vì viết code từ đầu cho mọi tính năng.



Mục tiêu chính:

- ✓ Giảm thời gian & chi phí phát triển
- ✓ Tối ưu nguồn lực cho vấn đề đặc thù
- ✓ Giảm thiểu sai sót

Các dạng "reuse" phổ biến:

-  Thư viện (Library Reuse)
-  Thành phần (Component Reuse)
-  Dịch vụ (Service Reuse)
-  Khung phần mềm (Framework Reuse)

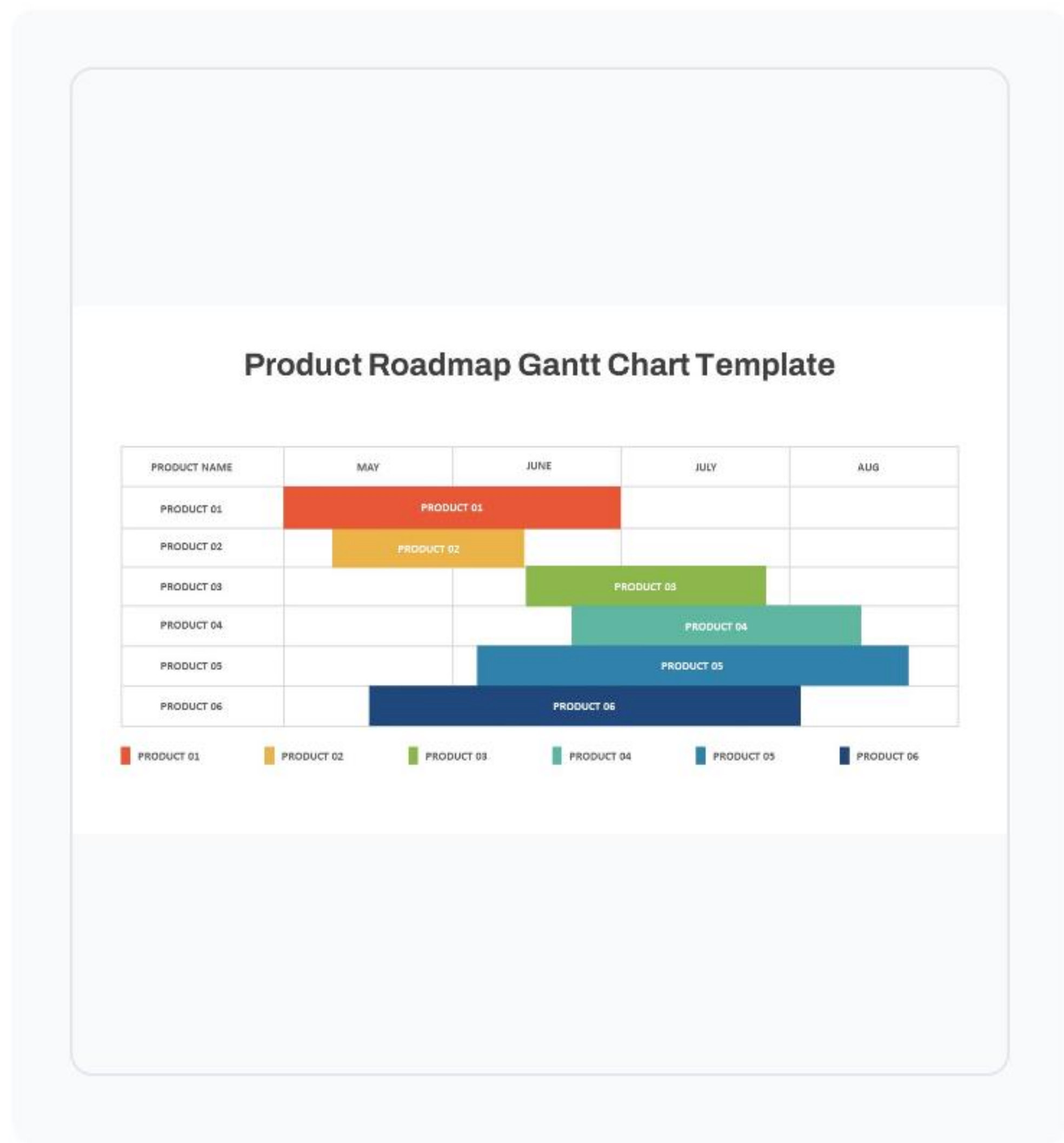
Mô hình Xoắn ốc (Spiral Model): Quản lý và Ứng dụng

Quản lý Rủi ro

- Đối đầu trực tiếp với rủi ro, không né tránh.
- Xác định sớm các vấn đề tiềm ẩn (công nghệ, nhân sự, yêu cầu).

4 Hoạt động Chính

1. Xác định mục tiêu (Planning)
2. Phân tích rủi ro (Risk Analysis)
3. Phát triển & Kiểm thử (Engineering)
4. Đánh giá & Lập kế hoạch tiếp theo (Evaluation)



Bản mẫu & Go/No-Go

- Xây dựng bản mẫu nhỏ để khách hàng dùng thử.
- Quyết định tiếp tục/dừng dự án dựa trên đánh giá rủi ro.

Dự án phù hợp

- Quy mô lớn, phức tạp, yêu cầu không rõ ràng.
- Công nghệ mới, rủi ro cao.
- Các dự án quân sự, hàng không, ngân hàng (độ an toàn tuyệt đối).

Mô hình Xoắn ốc (Spiral Model): Giải quyết rủi ro

⚠ Vấn đề mô hình Xoắn ốc giải quyết

Mô hình Xoắn ốc được phát triển để giải quyết những hạn chế của các mô hình truyền thống, đặc biệt trong các dự án phức tạp.

Không chắc chắn

Phù hợp khi yêu cầu khách hàng chưa rõ ràng hoặc dự án sử dụng công nghệ mới, giảm thiểu các rủi ro không lường trước.

Rủi ro cao

Tập trung vào quản lý rủi ro xuyên suốt chu kỳ phát triển, giúp xác định và giải quyết sớm các mối đe dọa tiềm tàng.

Quy mô lớn

Hỗ trợ các dự án lớn, cho phép nhìn nhận và phát triển từng phần, giảm sự phức tạp tổng thể.

🔄 Bốn hoạt động chính trong mỗi vòng xoắn

1. Xác định Mục tiêu (Planning)

Đặt mục tiêu cho vòng xoắn, lựa chọn giải pháp thay thế và các ràng buộc (chi phí, thời gian).

2. Phân tích Rủi ro (Risk Analysis)

Đánh giá các rủi ro tiềm ẩn. Là 'linh hồn' của mô hình, giúp quyết định dừng hay tiếp tục dự án.

3. Phát triển & Kiểm thử (Engineering)

Thực hiện thiết kế, viết mã, xây dựng bản mẫu (Prototype) và kiểm thử tính năng của vòng xoắn đó.

4. Đánh giá & Lập kế hoạch (Evaluation)

Khách hàng xem xét kết quả, nhóm phát triển cùng nhau lên kế hoạch cho vòng xoắn kế tiếp.

Mô hình RUP (Rational Unified Process): Linh hoạt và khác biệt

Quy trình lặp (Iteration) trong RUP

- RUP khuyến khích phát triển theo chu kỳ lặp (iterative) và gia tăng.
- Mỗi vòng lặp thực hiện đầy đủ các bước: Phân tích, Thiết kế, Code, Test.
- Mục tiêu: có phiên bản hoàn thiện hơn sau mỗi lặp, phát hiện lỗi sớm.

4 Pha chính của RUP



RUP so với Waterfall: Điểm khác biệt

Đặc điểm	Waterfall	RUP
Cấu trúc	Tuyến tính, một chiều	Lặp lại & tăng dần
Quản lý rủi ro	Lộ diện cuối dự án	Giải quyết sớm (Elaboration)
Sản phẩm	Chỉ thấy khi kết thúc	Phiên bản chạy được sau mỗi vòng lặp
Tài liệu	Nặng nề giấy tờ	Tập trung mô hình hóa

Vì sao RUP 'cấu trúc nhưng linh hoạt'?

- Có cấu trúc:** Quy định rõ ràng vai trò, hoạt động, sản phẩm bàn giao.
- Linh hoạt:** Cơ chế lặp cho phép điều chỉnh yêu cầu mà không phải làm lại.

Mô hình RUP (Rational Unified Process): Cấu trúc

Tổng quan về RUP

RUP (Rational Unified Process) là một phương pháp phát triển phần mềm có cấu trúc, lặp lại (iterative) và tăng dần (incremental), nhằm tạo ra sản phẩm chất lượng cao.

Được coi là quy trình quản lý dự án Agile, RUP chia dự án thành 4 pha chính. Mỗi pha bao gồm nhiều vòng lặp nhỏ, giúp giải quyết rủi ro sớm và thích nghi với thay đổi.

Bốn pha chính của RUP

Pha Khởi đầu (Inception)

Xác định phạm vi, ước tính chi phí và đánh giá rủi ro. Mục tiêu: 'Dự án này có đáng làm không?'

Pha Thiết kế chi tiết (Elaboration)

Tập trung xây dựng Kiến trúc để giảm thiểu rủi ro kỹ thuật.

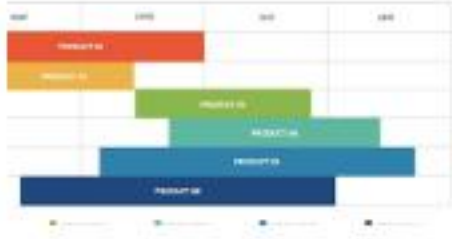
Pha Xây dựng (Construction)

Giai đoạn lập trình và tạo tính năng chính. Kết thúc pha là sản phẩm chạy được (Bản Beta).

Pha Chuyển giao (Transition)

Bàn giao sản phẩm, đào tạo, sửa lỗi cuối cùng và triển khai thực tế.

Image Sources



<https://www.slidekit.com/wp-content/uploads/2024/08/PowerPoint-Template-For-Product-Roadmap-Gantt-Chart-Template.jpg>

Source: www.slidekit.com